Software Requirements and Architecture

SWENG 894 Capstone

MicroMatch

Chastidy Joanem

1. Software Requirements

This section defines the functional and non-functional software requirements for the MicroMatch system. These requirements are derived directly from the Concept of Operations (CONOPS) and refine the initial feature set into testable, implementation-ready specifications. The requirements are expressed using a formal “the system shall” format to ensure clarity, traceability, and alignment with system goals.

1.1 Functional Requirements (Formal Specifications)

The following functional requirements define the core behavior of the MicroMatch MVP. As a solo project, 15 functional requirements are defined.

FR-1 The system shall allow users to create an account using an email address and password.

FR-2 The system shall support role-based access control for Student, Organization, and Administrator users.

FR-3 The system shall allow students to create and edit a profile, including skills, interests, and experience.

FR-4 The system shall allow organizations to create and manage organization profiles.

FR-5 The system shall allow organizations to create micro-internship listings with required skills, estimated effort, deliverables, deadlines, and compensation type.

FR-6 The system shall allow students to browse and search for available micro-internship listings.

FR-7 The system shall allow students to apply to micro-internships.

FR-8 The system shall track application status changes (applied, accepted, rejected, completed).

FR-9 The system shall allow organizations to accept or reject student applications.

FR-10 The system shall allow students to submit project deliverables via file upload or external links.

FR-11 The system shall allow organizations to review submitted deliverables.

FR-12 The system shall allow organizations to mark projects as completed.

FR-13 The system shall allow both students and organizations to submit feedback upon project completion.

FR-14 The system shall allow administrators to moderate users and project listings.

FR-15 The system shall log system actions for auditability and traceability.

2. Product Backlog

The product backlog represents all known functional and architectural requirements for the MicroMatch system. Functional requirements are mapped to User Stories to support Agile planning and prioritization.

2.1 User Stories

Each functional requirement is represented as a user story in the product backlog.

ID	User Story
US-1	As a user, I want to register and log in so that I can access the platform.
US-2	As a student, I want to create a profile so that organizations can evaluate my skills.
US-3	As an organization, I want to create a profile so students can learn about my mission.
US-4	As an organization, I want to post micro-internships so I can find short-term help.
US-5	As a student, I want to browse and search for projects so I can find relevant opportunities.
US-6	As a student, I want to apply to projects so I can gain experience.
US-7	As an organization, I want to review applications so I can select candidates.
US-8	As a student, I want to track my application status so I know where I stand.
US-9	As an organization, I want to accept or reject applications to manage my projects.
US-10	As a student, I want to submit deliverables so I can complete my project.
US-11	As an organization, I want to review deliverables so I can validate completion.

US-12	As a user, I want projects marked complete so work is formally closed.
US-13	As a user, I want to provide feedback so skills and performance are documented.
US-14	As an admin, I want to moderate content to maintain platform quality.
US-15	As a system, I want to log actions so activities are auditable.

3. Sprint Backlog

The Sprint Backlog includes the most critical and foundational user stories required to begin building a functional MVP. Stories are prioritized based on dependency, user value, and implementation effort.

3.1 Estimation Method

User Stories are estimated using Story Points (Fibonacci sequence). Priority is defined as High, Medium, or Low.

3.2 Sprint 1 Backlog (Foundational MVP)

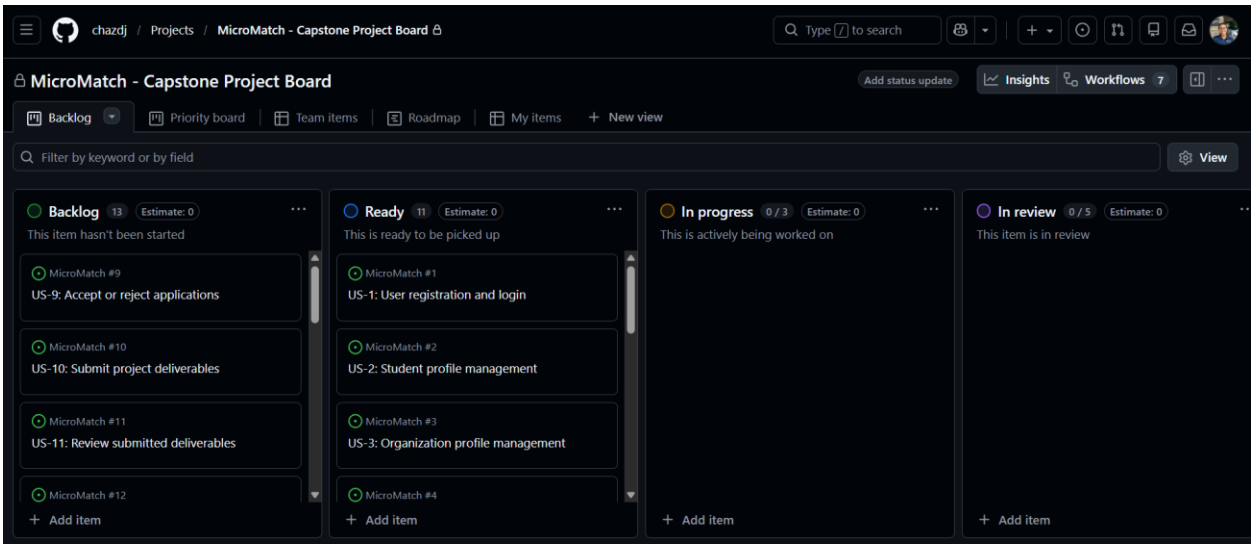
User Story	Priority	Story Points	Rationale
US-1 User Registration/Login	High	5	Required for all functionality
US-2 Student Profile	High	5	Core user data
US-3 Organization Profile	High	5	Required to post projects
US-4 Post Micro-Internship	High	8	Core marketplace functionality
US-5 Browse/Search Projects	High	5	Enables discovery
US-6 Apply to Projects	High	5	Core interaction
US-7 Review Applications	Medium	5	Depends on applications

US-8 Track Application Status	Medium	3	Enhances usability

These items form the minimum viable foundation to support end-to-end project flow.

4. Agile Board and Tooling

Both the Product Backlog and Sprint Backlog are managed using GitHub Projects in a Kanban-style board. The board includes columns for Backlog, Ready, In Progress, Review, and Done. All user stories and architectural tasks are tracked as issues and linked to commits for traceability.



[GITHUB PROJECT LINK]

5. Non-Functional Requirements (Quality Attributes)

The MicroMatch architecture is designed to support the following quality attributes:

5.1 Security

- User authentication and role-based authorization
- Secure handling of user data and uploaded files
- API access protected via token-based authentication

5.2 Scalability

- Stateless backend services using FastAPI

- PostgreSQL used for relational integrity and scaling
- Frontend and backend deployed independently

5.3 Maintainability

- Modular frontend components in React
- Layered backend architecture (API, service, persistence)
- Automated testing using Pytest and React Testing Library

5.4 Usability

- Clear workflows for students and organizations
- Minimal steps to apply and manage projects
- Responsive web interface

6. Architectural Requirements and Design

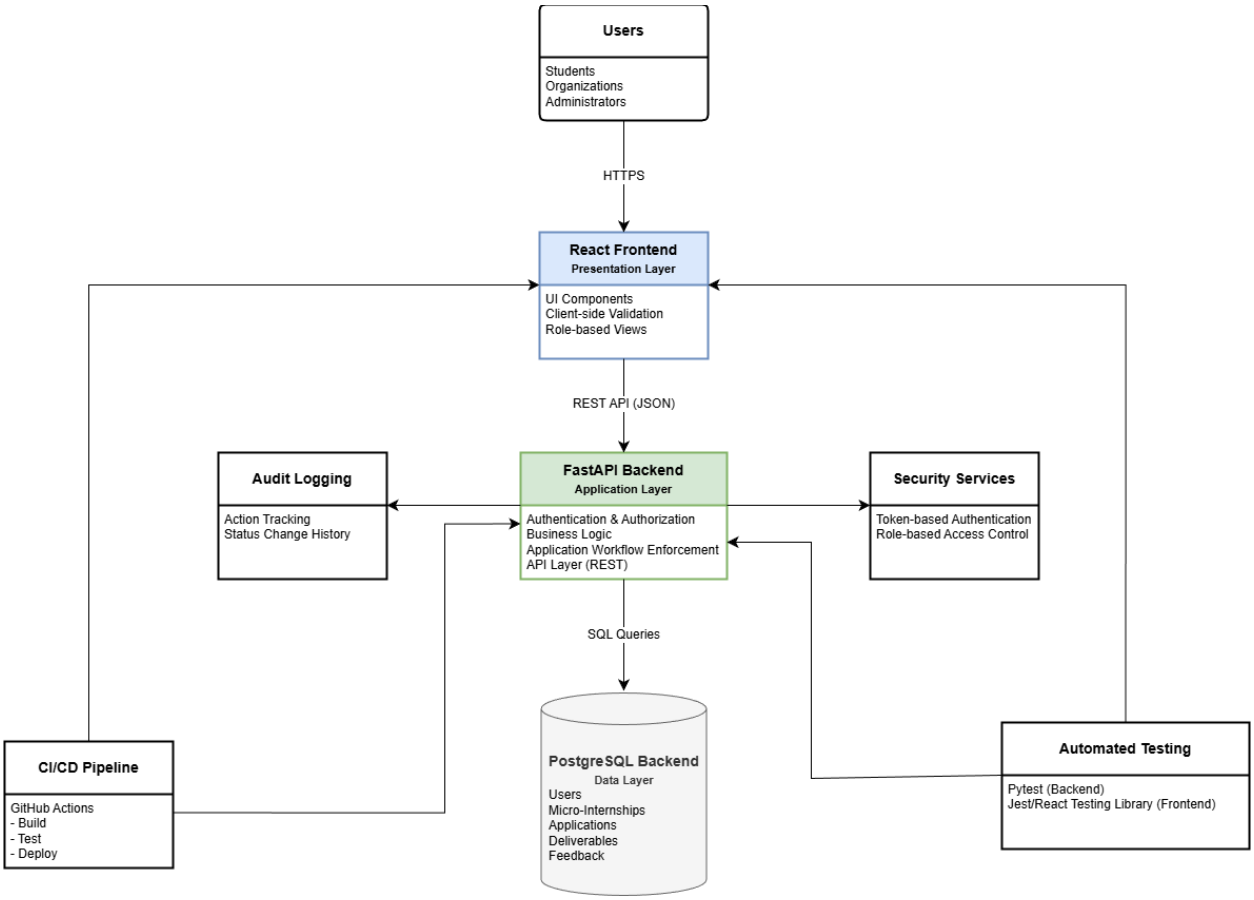


Figure 2. MicroMatch High-Level Architecture Diagram

6.1 High-Level Architecture

MicroMatch follows a three-tier web architecture:

- **Presentation Layer:** React-based frontend
- **Application Layer:** FastAPI backend REST services
- **Data Layer:** PostgreSQL relational database

6.2 Component Responsibilities

Frontend (React)

- User interface rendering
- Form validation
- API interaction

Backend (FastAPI)

- Business logic
- Authentication and authorization
- Application workflow enforcement

Database (PostgreSQL)

- Persistent storage of users, projects, applications, and feedback
- Referential integrity and constraints

CI/CD (GitHub Actions)

- Automated testing
- Build and deployment pipelines

6.3 Architectural Rationale

This architecture was selected to:

- Support rapid development and iteration
- Separate concerns for maintainability
- Align with industry-standard web application patterns
- Enable scalability without premature complexity

The architecture directly supports the identified non-functional requirements, particularly scalability, maintainability, and security.

7. Conclusion

This document refines the MicroMatch project vision into actionable software requirements and an initial architecture. The defined product and sprint backlogs provide a clear development roadmap, while the architecture supports both current MVP goals and future extensibility. Together, these artifacts establish a strong foundation for iterative development throughout the remainder of the capstone project.